

把碎片化数据喂给 LLM 会得到一个困惑的 LLM What is AI-ready data? How a knowledge layer gets you there

本篇范围声明：这是 Neo4j 官方博客的一篇**长篇指南**（标注阅读时长 **15 分钟**，是这两周读到的 Neo4j 文章里最实在的一篇）。它带 Gartner 与 IDC 的引用数字、两个生产案例、以及一套五步法。下面的「全文翻译」覆盖正文全部六节（AI-ready 的三个质量、为什么需要知识层、三个组件、三项收益、两个案例、五步法）；FAQ 与页脚不译。它**仍然是营销材料**，结尾导向产品，但主体的结构和数据可以单独取用。

核心内容

Q：这篇的问题陈述是什么？ A：大多数企业 AI 项目卡在生产环节，而背后的数据通常就是原因。

Without AI-ready data, AI systems have access to your information but no way to reason over it. 没有 AI-ready 的数据，AI 系统能访问你的信息，但没有办法在其上推理。

根因是架构错配：

传统企业数据基础设施	AI 系统
为谁建而结果 — 确定性软件：固定输入 → 固定工作流 → 固定输出，全在 刚性模式与孤岛系统 之内	你给 agent 一个目标，它自己决定拉什么数据、自己规划步骤、自己行动
	大多数企业数据今天支撑不了这种非确定性的自主性

两个引用数字：

- Gartner：63% 的组织要么没有、要么不确定自己有适合 AI 的数据管理实践
- Gartner 预测：到 2026 年，组织将放弃 60% 缺乏 AI-ready 数据的 AI 项目

Q：什么算 AI-ready？三个质量。

AI-ready data is data that AI agents can use to reliably reason, decide, and act on to produce accurate, explainable, and governable outcomes. AI-ready 的数据，是 AI agent 能用

来可靠地推理、决策和行动，从而产出准确、可解释、可治理结果的数据。

它归结为三个质量：有上下文（contextual）、可扩展（flexible）、口径统一（standardized）。

一、有上下文 —— 不只把单个事实暴露给 agent，还暴露它们彼此的连接。

反例是一个处理退款请求的客服 agent：没有上下文时，它孤立地取出订单详情，然后照着规则手册走一遍，输出好一点的泛泛而谈、坏一点的错的。有上下文时，同一个 agent 沿连接取出订单、这位客户的购买模式、他的客服历史、他的合同条款、以及过去类似案例是怎么解决的——这时它才能针对这个具体客户的处境推理。

二、可扩展 —— 数据随 AI 用例演化而适应，让 agent 能拿到新信号而不用等数据库重构。

反例讲得很细：你上线一个标记流失风险客户的 agent，两个月后业务方想让它再考虑「哪些客户最近换了客户经理」和「哪些订阅档位快到期」——两个你原本没建模的信号。

没有可扩展的数据，加这两个信号意味着跨 customer、account、subscription 三张表做模式迁移，加上重写流失流水线里的每一条查询。改进版 agent 下个季度上线，不是下个 sprint 上线。

有可扩展的数据，团队只是往模型里加两个新标签、连到已有实体上，agent 通过连接就拿到了新信号。

「AI 计划按你的用例节奏推进，而不是等数据库赶上来。」

三、口径统一 —— 同一个含义横跨所有来源保持一致。

这个反例是全文最该记的一个：一个 agent 被要求总结某客户的近期活动，它要

- 从 CRM 取客户记录（那里的 ID 是字符串）
- 从计费系统取最新发票（同一个人在那里的 ID 是整数）
- 取一张最近的客服工单（UUID）

The agent has to guess which records belong to the same person — adding latency, cost, and the risk of assembling a summary from data that doesn't actually belong to one customer. That mistake feeds straight into your downstream decisions. agent 只能猜哪些记录属于同一个人——这带来延迟、成本，以及用「实际上不属于同一个客户的数据」拼出一份摘要的风险。那个错误会直接流进你的下游决策。

有口径统一的数据，客户有一个规范定义（canonical definition），其余所有客户数据通过关系映射到这个规范定义上。

Q：知识层由哪几个组件构成？

知识层坐在你现有的数据基础设施和你的 AI 系统之间。三个组件：

一、知识图谱 —— 长期记忆

它从结构化和非结构化来源拉数据，**解析实体（让同一个实体被一致地识别）**，映出实体间的连接，**用反映你业务本体的元数据丰富实体和关系**，把这一切存成一张可导航的图。

「这成为你的 AI 的长期记忆。」

二、上下文图 —— agent 的情境记忆

业务知识只能让 agent 走到一半。要长期可靠运行，agent 还需要记住**它决定了什么、行动了什么、学到了什么**。

上下文图在知识图谱之上扩展**两种额外的记忆**：

记忆类型	装什么
短期记忆	进行中的对话：用户刚问了什么、有哪些任务是开着的、这一轮 agent 收集了什么上下文
推理记忆（reasoning memory）	agent 的决策历史：它试过的计划、调用过的工具、观察到的结果、以及第一次尝试失败后你给它的纠正

Every decision becomes a traceable path through the graph. Short-term and reasoning memory live in the same graph as your business knowledge, connected by explicit relationships. Every decision and action can be replayed, audited, and learned from. 每个决定都成为图中一条可追溯的路径。短期记忆和推理记忆与业务知识住在同一张图里，由显式的关系连接。每个决定和行动都能被回放、审计、并从中学习。

长期记忆 + 短期记忆 + 推理记忆 = 任何生产级 AI 系统所需的完整 agent 记忆。

三、GraphRAG —— 图检索与多步推理

标准的向量 RAG 按**语义相似度**取文本块。对直白的问题，答案可能就出现在某个相似段落里。但**依赖多步推理的复杂问题**——比如「**基于订单历史，这个客户的供应商风险是什么？**」——可能匹配不上任何单个文本段落。

AI 需要：**查这个客户 → 看他的订单和产品 → 把那些产品连到供应商 → 评估供应商的交付历史**，才能给出有依据的答案。

GraphRAG 让 agent 沿关系链多跳遍历，找到全部相关信息，即使它们语义上并不相似。

Q：三项收益各有什么依据？

一、更准确的答案 —— 有数字

A May 2026 IDC study found that grounding AI in Neo4j knowledge graphs reduced hallucination rates by an average of 44%, with one life sciences customer reducing hallucination rates from 20-40% to 2-5%. 2026 年 5 月的一项 IDC 研究发现，把 AI 锚定在 Neo4j 知识图谱上平均降低幻觉率 44%，其中一个生命科学客户把幻觉率从 20 - 40% 降到 2 - 5%。

另有 NICD 的研究：[向量 RAG vs 简单 GraphRAG](#)，后者准确度 2 倍。

二、可解释的推理 —— agent 取上下文时走的路径被记录下来：它取了哪些实体、遍历了哪些关系、调用了哪些工具、哪些事实支撑了这个答案。

三、可治理的 AI —— 这一条有一句锋利的話

Governance cannot depend on prompts alone. 治理不能只依赖提示词。

知识层让**定义、血缘、权限、规则和决策轨迹**在**数据架构里显式化**。团队能控制 agent 可访问哪些数据、追溯它用了什么、审计输出是怎么产生的。

并且：随着用例增长，同一张图能加新实体和新关系而演化，不迫使团队从头重建治理模型。

Q：两个生产案例。

Klarna（金融科技）：组织知识散在**超过 1200 个 SaaS 系统里**，包括 Salesforce 和 Workday。碎片化让 AI 的回答难以锚定。

它建了内部 AI 助手 **Kiki**（Neo4j 知识图谱 + OpenAI 大模型），连接人、项目、流程、系统的知识。

结果：回答了超过 25 万次询问、85% 的员工在用、Klarna 废弃了 1200 多个 SaaS 系统。

Klarna 的那句引语是全文最好记的一句：

「Feeding an LLM the fractioned, fragmented, and dispersed world of corporate data will result in a very confused LLM.」 「把那个割裂、碎片、分散的企业数据世界喂给一个 LLM，结果会是一个非常困惑的 LLM。」

Data²（国防、情报、能源数据）：有价值的信息常住在**钻井报告、设施记录、维护日志**这类非结构化来源里，关系型数据库难以跨关系分析。

它建了 **reView** 平台，知识图谱建在 Neo4j AuraDB 上，用 GraphRAG 做问答与证据审阅，采用分离的「**证据图**」和「**问题图**」，帮助防止语言模型摄入错误数据。分析师能钻取每个答案背后的证据，**跟着看系统是怎么得出结论的**。

Data² 估计 reView 释放了 50% 的分析师工作量。

Q: 五步法是什么？

作者先给了一条重要的建议：

Start with one AI use case and the data it needs, then go from there. This practical sequence is more useful than a broad data modernization program. 从一个 AI 用例和它需要的数据开始，然后往下推。这个实用的顺序比一个宽泛的数据现代化项目更有用——因为它让你围绕业务真正需要被回答的一个问题去建模实体、关系、上下文和检索模式。

步	内容
1. 定义用例与图 schema	选定 AI 系统要支撑的那个问题或任务。定义支撑这个用例的数据 schema：有哪些实体、它们怎么互相关联、有哪些属性。决定什么算一个客户、客户怎么关联到订单/供应商/工单/合同/客户经理、以及哪些定义、规则和权限适用。围绕用例建模，然后随更多问题浮现再扩展 schema
2. 从结构化与非结构化源建知识图谱	流水线要抽实体、去重、加关系、加元数据、入图。不要试图一次把整个企业建模完，从解决用例的高价值数据开始
3. 索引数据并设置检索模式	不同问题需要不同的检索路径，多数生产系统会用不止一种。更强的检索设置把每个问题路由到正确的模式，而不是每次都查所有来源。向量或全文索引帮 agent 找准入口点，Cypher 查询模板帮它更快更可靠地走常见路径
4. 建上下文图让 AI 能学	生产 AI 需要有状态的记忆：记录任务期间发生了什么，包括取到的证据
5. 评估用例是否成功并改进	—

背景补充 / 点评

「口径统一」那个 ID 反例，是这篇给我价值最大的一段

三个系统里同一个人有三种 ID（字符串、整数、UUID），agent 只能猜哪些记录属于同一个人。

这一段之所以有用，是因为它把「定义不一致」这个抽象问题落到了一个具体后果上：

那个错误会直接流进你的下游决策。

而且它指出了正确的解法形状：不是让 agent 在运行时去拼身份（stitching identities together at runtime），而是让「客户」有一个规范定义，其余所有客户数据通过关系映射到它。

「运行时拼身份」这个说法很准确——它说的是一个成本被推到了错误的位置：本该在建模时解决一次的事，变成了每次查询都要解决一遍，而且每次都可能解错。

「推理记忆」是这篇提出的最有意思的概念

三层记忆的划分（长期 / 短期 / 推理）里，前两层是常见说法，第三层不常见：

推理记忆捕获 agent 的决策历史：它试过的计划、调用过的工具、观察到的结果、以及第一次尝试失败后你给它的纠正。

注意最后一项——「你给它的纠正」。这意味着推理记忆不只记 agent 自己做了什么，还记人在什么地方否定了它。

而且这三层住在同一张图里、由显式关系连接，所以「每个决定都成为图中一条可追溯的路径」。

这一条的分量在于它让「回放和审计」变成了图遍历，而不是去翻日志。

「治理不能只依赖提示词」

这句话短，但它划出了一条清楚的界：

靠提示词做治理	靠架构做治理
「不要引用未经批准的文档」写在 system prompt 里	权限在数据架构里，agent 根本访问不到那些数据
「回答时要给出依据」写在 prompt 里	遍历路径被自动记录，依据是副产品不是要求
模型可能忘、可能被绕过	绕不过，因为它不是一条指令而是一个约束

作者列的五样要在数据架构里显式化的东西：定义、血缘、权限、规则、决策轨迹。

五步法第 1 步的措辞值得逐字读

决定什么算一个客户（Decide what counts as a customer）、客户怎么关联到订单/供应商/工单/合同/客户经理、以及哪些定义、规则和权限适用。

「决定什么算一个客户」这句话把一件常被跳过的事摆到了第一步——它不是技术动作，是裁决动作。而且它必须在建 schema 之前完成，不能边建边定。

紧接着那句「围绕用例建模，然后随更多问题浮现再扩展 schema」是另一半：不要求一次定完，但要求每次定的那部分是定死的。

两个案例里，Data² 的那个架构细节值得单独拎出来

采用分离的「证据图」和「问题图」，帮助防止语言模型摄入错误数据。

这是全文唯一一处具体的架构模式，而且它解决的是一个真问题：如果证据和问题在同一张图里，模型在遍历时可能把「别人问过的问题」当成「事实」读进去。分成两张图，物理上隔开了这两类东西。

文中没有更多细节（两张图怎么连、查询时怎么协同），但这个模式本身值得记。

我认为要打折扣的四处

第一，那些数字要看清出处和口径。

数字	出处	要注意什么
幻觉率平均降低 44%	IDC 2026 年 5 月 IDC 是分析师机构，这类研究常由厂商赞助；文中没说样本研究	量、没说对照组是什么
一个客户从 20 - 40% 降到 2 - 5%	同上	单个案例，而且原始区间宽到 20 - 40%，本身说明测量口径不严
GraphRAG 准确度 2 倍	NICD	是一个独立机构，但文中只给结论不给条件
Gartner 63% / 60%	Gartner	「不确定自己有」被算进 63% 里，这个口径会放大数字

第二，Klarna 那个「废弃了 1200+ 个 SaaS 系统」不能归因给知识图谱。一家公司同时有 1200 个 SaaS 系统本身就是治理失控，废弃它们主要是采购和流程整合的结果，图谱最多是让整合过程看得清。把它列成图谱的成果是过度归因。

第三，五步法里第 5 步几乎是空的。「评估用例是否成功并改进」——用什么指标评估、成功的阈值是多少、改进的循环怎么闭合，一个字都没有。而这一步恰恰是最难的（这正是我昨天在三份 OKR 上花一整天处理的那件事）。

第四，「可扩展」那个卖点被简化了。「只是往模型里加两个新标签、连到已有实体上」——这在图数据库里技术上确实容易，但它没说的是：加了标签之后，谁保证这两个新标签的含义和已有的不冲突？无约束的「加标签就行」正是 9/4 读到的那个问题——RDF 层不认识类、个体、属性的区别，谁都能当主语，灵活性的代价是要另加校验去守。

对我的启示

第一条，「运行时拼身份」这个说法可以直接用来描述 prove 现在的一个状态。

那个 ID 反例说的是：本该在建模时解决一次的身份统一，变成了每次查询都要解决一遍。

对照 prove: mapping-registry.yaml 登记了 29 个实体各自对应哪张表哪几列，这就是「规范定义 + 关系映射」那条正确路径。但有一处没做到——title 被四个实体共用（course_dim、session_dim、

`participation`、`session_completion`)，而判断「这四个 title 是不是同一个含义」目前靠人，没有机器检查。

基石 B3 写了「一个名字一个含义：同名必同义，同义必同名」，但它的落地形态是什么？这篇提示了答案：要有一个规范定义，其余映射到它——也就是说 `title` 应该有一个单独的登记条目说明它的含义，四个实体的 `fields` 引用那个条目，而不是各自写一遍 `type: str`。

第二条，「推理记忆」正是我这几天手工在做、但没有机制的那件事。

我在四份本体文档里记的东西——属性和关系的分界改了三次、`rdfs:domain` 被撤回、B8 的表述被修正、J6 四问的确立、以及每次崔扬否决了什么——按这篇的划分，全部属于「推理记忆」：试过的方案、失败的原因、人给的纠正。

而它们现在只在 markdown 里，不在任何可查询的结构里。

这一条和 9/6 读的 Document Intelligence 路线图里的「Context graph：模式的来源、血缘、以及构建时做过的决策，被捕获成一张独立的图」是同一件事。两篇独立提到同一个缺口，说明它不是我一个人的洁癖。

具体能做的一小步：`umu:kind` 是我这几天加的第一个把建模选择显式登记进图的机制，但它只记了结果（是属性还是关系），没记理由和时间。可以考虑加两个注解字段：`decided_at`（哪天定的）和 `decided_by`（谁拍的板），这两个都是注解、在逻辑外，符合 B9。

第三条，「治理不能只依赖提示词」这条界线，可以直接用来说明为什么选 SHACL 而不是 prompt。

我在合规检查评估里比较过三条路线，当时的理由是「可审查」。这篇给了更准的表述：

写在 prompt 里的规则，模型可能忘、可能被绕过；写成约束的规则，绕不过——因为它不是一条指令，而是一个约束。

这正是基石 B5「校验对数据报违规，不改数据」和「公理是检查单不是推理前提」的动机的另一种说法。

第四条，五步法第 1 步那句「决定什么算一个客户」，是我这几天做的事的准确名字。

我这周花的时间几乎全在裁决上——「谓词」禁用、「概念」改「类」、「取值域」改「词表类」、属性和关系的分界由应用按 J6 定——建模只是最后一步。

这篇把这件事放在五步的第一步，而且明确它必须在建 schema 之前完成。这给了我一个可以引用的顺序依据：裁决先于建模，不是建模过程中顺带解决的。

第五条，「不要试图一次把整个企业建模完」这条建议，和我给 prove 的一个观察对上上了。

prove 现在有 34 个类、53 条关系、17 个词表类，但没有一个「这套本体建成之后应该能回答哪些问题」的清单（我 9/6 记过这条待办）。

按这篇的顺序，这个清单本该是第 1 步的产物。没有它，本体的边界就是按「数仓有什么」定的，不是按「业务要问什么」定的——而后者才是这篇说的「围绕用例建模」。

文中金句

"Without AI-ready data, AI systems have access to your information but no way to reason over it." 没有 AI-ready 的数据，AI 系统能访问你的信息，但没有办法在其上推理。

"Feeding an LLM the fractioned, fragmented, and dispersed world of corporate data will result in a very confused LLM." (Klarna) 把那个割裂、碎片、分散的企业数据世界喂给一个 LLM，结果会是一个非常困惑的 LLM。

"The agent has to guess which records belong to the same person ... That mistake feeds straight into your downstream decisions." agent 只能猜哪些记录属于同一个人……那个错误会直接流进你的下游决策。

"Governance cannot depend on prompts alone." 治理不能只依赖提示词。

"Every decision becomes a traceable path through the graph. Every decision and action can be replayed, audited, and learned from." 每个决定都成为图中一条可追溯的路径。每个决定和行动都能被回放、审计、并从中学习。

"The AI initiative moves at the pace of your use cases, instead of waiting for your databases to catch up." AI 计划按你的用例节奏推进，而不是等数据库赶上来。

"Start with one AI use case and the data it needs ... This practical sequence is more useful than a broad data modernization program." 从一个 AI 用例和它需要的数据开始……这个实用的顺序比一个宽泛的数据现代化项目更有用。

全文翻译

开篇：为什么企业 AI 卡在生产

大多数企业 AI 项目卡在生产环节，而背后的数据通常就是原因。没有 AI-ready 的数据，AI 系统能访问你的信息，但没有办法在其上推理。

传统的企业数据基础设施是为确定性软件建的：拿一个固定输入、走一条固定工作流、返回一个固定输出，**全都在刚性的模式与孤岛式的系统之内。**

AI 系统的工作方式不同。你给一个 AI agent 一个目标，它自己决定拉什么数据、自己规划步骤、自己行动。但今天大多数企业数据支撑不了那种非确定性的自主性。

（Gartner 的两个数字见「核心内容」。）

要把 AI 举措推进到生产的组织，需要它们能构建、维护、并信任的 AI-ready 数据。知识层通过提供「AI 能导航、理解、并推理」的数据来提供那个基础。

一、什么是 AI-ready 数据

（定义与三个质量的完整内容见「核心内容」。）

二、为什么 AI-ready 数据需要一个知识层

想想你自己是怎么做决定的。你的大脑不是把事实存在孤立的行里。你把多个事实连到上下文上（当前处境、过往经验、相关知识）才得出一个好决定。AI 系统也需要一个结构，帮它建立这些连接、并以同样的方式在数据上推理。

一个知识层坐在你现有的数据基础设施和你的 AI 系统之间，从而让你的数据为 AI 准备好。图为这个知识层提供结构，让 agent 能导航、理解、并推理你的数据。

一个 AI-ready 的架构用知识层把现有系统、检索、和 agent 记忆连接进一个受治理的基础。具体是：

- 从现有的运营数据库和非结构化内容库摄入数据进企业知识图谱，抽实体，用向量 embedding 索引它们以支持快速检索
- 用虚拟图（virtual graphs），系统在知识图谱的上下文里读你的数据仓库，从而在不需要迁移的前提下派生智能
- agent 把自己的对话与决策历史写进图作为额外上下文，把一切连到你已有的知识上
- AI 随后从这个知识层推理，而这个知识层坐在你当前的数据基础设施之上

（三个组件的完整内容见「核心内容」。）

三、知识层给 AI 带来的收益

通过用一张互连的图取代孤立的数据，你给 AI agent 提供了它们所需的知识层，用于整体性的上下文和复杂推理。这个架构转变的结果是：AI 的产出更准确、更可解释、更可治理。

（三项收益与数字见「核心内容」。）

四、案例

看数据管理实践，AI-ready 的企业有什么共同点？

They stop trying to move all data into one warehouse or database and build a knowledge layer that maps entities, relationships, and meaning across sources. 它们停止试图把所有数据搬进一个数仓或数据库，转而建一个知识层，跨来源映出实体、关系与含义。

(Klarna 与 Data² 两个案例的完整内容见「核心内容」。)

五、让数据 AI-ready 的五步

(完整五步与那条「从一个用例开始」的建议见「核心内容」。)

关于第 3 步作者多说了一句：[不同问题需要不同的检索路径，而多数生产系统会用不止一种。一个更强的检索设置把每个问题路由到正确的那个模式，而不是每次都去查所有来源。](#)

关于第 4 步：[生产 AI 需要有状态的记忆](#)，它需要记录任务期间发生了什么，包括取到的证据。

参考链接

- [What is AI-ready data? How a knowledge layer gets you there](#) — 本文原文 (Neo4j 博客, 15 分钟长文)
- [Gartner: Data management practices for AI](#) — 63% 与 60% 两个数字的出处方
- [IDC study on Neo4j knowledge graphs \(2026-05\)](#) — 幻觉率平均降低 44% 的出处，注意这类研究常由厂商赞助
- [GraphRAG makes AI agents 80% more truthful — NICD](#) — 向量 RAG vs GraphRAG 对比研究
- [Klarna 案例 \(Kiki 内部 AI 助手\)](#) — 1200+ SaaS、25 万次询问、85% 员工使用
- [Data² reView 案例](#) — 分离的证据图与问题图，释放 50% 分析师工作量
- [Neo4j AuraDB](#) — Data² 的图谱所建之上